

MaskImpute: A Biozero-Aware Masked Autoencoder for Single-Cell RNA-seq Imputation

Anonymous Author(s)

Affiliation

Address

email

Abstract

Single-cell RNA sequencing (scRNA-seq) matrices are sparse and noisy, and observed zeros conflate biological non-expression with technical under-detection. We propose MaskImpute, a masked denoising autoencoder that uses different corruption schemes for observed zeros and non-zeros, a count-derived biozero prior as a soft regularizer, and reconstruction-informed shrinkage on observed zeros. On a Splatter simulation suite of 13 scenarios with known pre-dropout ground truth, MaskImpute reduces mean MSE by 38% relative to DCA and 35% relative to scVI, with corresponding dropout-MSE reductions of 34% and 32%, while keeping mean Biozero-MSE below both neural baselines. The benchmark reports full-matrix, dropout-focused, biozero-focused, marker-gene, and per-gene normalized reconstruction metrics in a fixed, dataset-independent configuration.

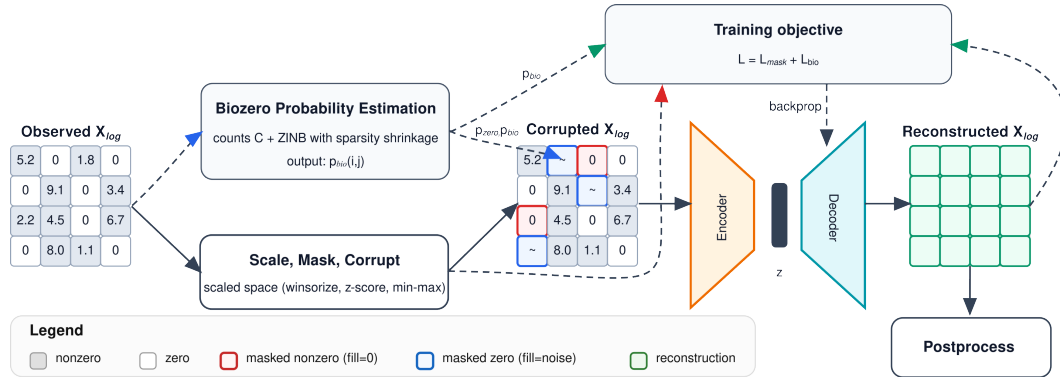


Figure 1: MaskImpute overview. Starting from logcounts X_{log} and counts C , we estimate a biozero probability $p_{bio}(i,j)$ for observed zeros (Section 3.5). During self-supervised training, non-zeros are masked at a fixed rate, while zeros are masked with probability proportional to p_{bio} ; masked zeros are corrupted by injecting gene-scale noise, and masked non-zeros are replaced by zero in the scaled space. A fully connected autoencoder reconstructs the scaled matrix and is trained with the objective $\mathcal{L} = \mathcal{L}_{mask} + \lambda_{bio}\mathcal{L}_{bio} + \lambda_{nz}\mathcal{L}_{nz}$ (Section 3.4). At inference, we invert the scaling to obtain $\hat{X}_{log}$ and apply reconstruction-informed shrinkage on observed zeros in the normalized count scale (Section 3.7).

1 Introduction

Single-cell RNA sequencing (scRNA-seq) enables large-scale measurement of RNA expression across individual cells and has become central to studying cellular heterogeneity, developmental trajectories, disease states, and perturbation responses [Luecken and Theis, 2019, Hicks et al., 2018, Van de Sande et al., 2023]. The resulting gene-by-cell matrix is sparse and noisy because only a small fraction of molecules are captured and sequenced, and additional technical factors (e.g., batch effects) perturb measurements [Brennecke et al., 2013, Hicks et al., 2018]. A central difficulty is the interpretation of zeros: an observed zero is a valid measurement that can arise from true biological non-expression or from limited molecular sampling (often termed “dropout”), and disentangling these sources from expression alone is generally not identifiable [Sarkar and Stephens, 2021, Svensson, 2020, Jiang et al., 2022, Linderman et al., 2022b]. This uncertainty complicates downstream analyses such as differential expression, trajectory inference, and cell-state interpretation [Hou et al., 2020, Luecken and Theis, 2019].

A large literature proposes denoising and imputation methods that treat scRNA-seq as a structured inference problem: borrow information across related cells and genes to reduce measurement noise. Graph diffusion methods such as MAGIC smooth expression by propagating information over a cell-cell neighborhood graph [van Dijk et al., 2018]; they can reveal global structure but may over-smooth within-state variability and inflate gene-gene correlations when diffusion is aggressive [Huang et al., 2018, Hou et al., 2020]. Probabilistic approaches such as SAVER and scImpute model aspects of the measurement process and/or dropout explicitly [Huang et al., 2018, Li and Li, 2018], but their performance depends on modeling assumptions and on how well “dropout” can be operationalized for a given protocol [Svensson, 2020, Jiang et al., 2022]. Deep autoencoder methods (e.g., DCA and AutoClass) scale to large datasets and learn nonlinear manifolds [Eraslan et al., 2019, Li et al., 2022], yet reconstructions are typically point estimates and can still blur rare states or distort variance if treated as ground truth [Hou et al., 2020, Linderman et al., 2022b].

We introduce **MaskImpute**, a masked denoising autoencoder designed around an uncertainty-aware view of zeros. Instead of treating every zero as a missing value to be filled, MaskImpute combines (i) a masking strategy with different corruption schemes for observed zeros vs. observed non-zeros, (ii) a count-derived biozero prior used only as a soft regularizer, and (iii) a reconstruction-informed shrinkage step applied only on observed zeros to reduce over-imputation while retaining recovery of likely technical under-detection.

This work makes the following contributions:

- We adapt masked self-supervised learning to scRNA-seq denoising with different masking/corruption schemes for zeros and non-zeros.
- We introduce a biozero-aware regularization and masking scheme that encourages zero-preserving behavior while still allowing recovery of likely technical under-detection.
- We introduce a reconstruction-informed post-hoc shrinkage mechanism that modulates imputed values only at observed zeros based on the model’s reconstructed signal.
- We benchmark MaskImpute against representative baselines on a synthetic suite with known ground truth, reporting full-matrix, dropout-focused, biozero-focused, marker-gene, and per-gene normalized errors.
- We provide a reproducible evaluation protocol that keeps MaskImpute settings fixed across synthetic scenarios and separates imputation accuracy from downstream task-specific post-processing.

2 Related Works

2.1 Statistical and Probabilistic Models

scRNA-seq denoising sits at the intersection of representation learning and uncertainty-aware inference: methods attempt to estimate latent expression under an imperfect measurement process, while avoiding the creation of spurious signal that can mislead downstream reasoning [Sarkar and Stephens, 2021, Hou et al., 2020, Linderman et al., 2022b].

Early methods focused on explicit probabilistic models of counts. SAVER [Huang et al., 2018] assumes a Poisson measurement model with a Gamma prior (negative binomial marginal) and uses

empirical Bayes regression to estimate gene- and cell-specific priors, producing posterior means and uncertainty estimates. scImpute [Li and Li, 2018] uses mixture modeling to estimate dropout probabilities and selectively imputes only entries deemed likely to be technical, borrowing information from similar cells. These approaches can be conservative and interpretable, but they depend on distributional assumptions and on how well the model captures protocol-specific noise (including the debated role of zero inflation) [Svensson, 2020, Jiang et al., 2022].

2.2 Graph-Based and Diffusion Methods

Graph-based methods assume that biologically similar cells lie on (or near) a low-dimensional manifold and that neighborhoods can be used to borrow signal across cells. MAGIC [van Dijk et al., 2018] constructs a k -nearest-neighbor affinity graph (often in a PCA space), normalizes it into a Markov transition matrix M , and imputes by diffusion, $\mathbf{X}' = M^t \mathbf{X}$, where t is the diffusion time. Such methods can be effective for visualization and smooth trajectories, but the global averaging implicit in diffusion can blur rare states and inflate correlations if neighborhood or diffusion parameters are too aggressive [Huang et al., 2018, Hou et al., 2020, Linderman et al., 2022b].

ccImpute [Malec et al., 2022] offers an alternative that builds a consensus similarity matrix from repeated K-means clustering (inspired by SC3 [Kiselev et al., 2017]) and imputes likely dropouts by consensus-weighted averaging over similar cells. Compared with diffusion, consensus approaches can reduce sensitivity to a single neighborhood choice, but they still inherit clustering biases and can sharpen boundaries; thus improvements in clustering metrics after imputation should be interpreted cautiously [Hou et al., 2020].

2.3 Deep Learning and Autoencoders

Autoencoder architectures scale to large datasets and can model nonlinear structure, making them attractive for scRNA-seq denoising. DCA [Eraslan et al., 2019] replaces a squared-error reconstruction loss with a count likelihood (NB/ZINB) so that the decoder outputs distribution parameters (e.g., mean and dispersion, and optionally zero-inflation). This can improve robustness to heavy-tailed count noise, but the interpretation of zero-inflation and per-entry dropout probabilities is limited by identifiability and by the ongoing debate over whether additional zero-inflation is needed for common UMI protocols [Svensson, 2020, Jiang et al., 2022]. More broadly, neural denoisers typically output point estimates that can be over-interpreted as “ground truth” and can distort variance or downstream statistics if used uncritically [Hou et al., 2020, Linderman et al., 2022b].

scVI [Lopez et al., 2018] is a variational autoencoder that models scRNA-seq counts with an explicit generative process and has become a common baseline for scalable representation learning and denoising. In contrast to neural approaches, ALRA [Linderman et al., 2022a] performs an adaptively-thresholded low-rank approximation that explicitly preserves zeros, providing a strong zero-preserving imputation baseline.

AutoClass [Li et al., 2022] augments an autoencoder with a classifier branch trained on pseudo-labels from pre-clustering, encouraging the bottleneck to preserve population structure. This can improve separability, but it introduces dependence on the pre-clustering signal and can propagate confounders (e.g., batch structure) present in pseudo-labels [Hou et al., 2020]. MaskImpute adopts a self-supervised masking objective to learn structure without requiring explicit pseudo-labels.

2.4 Self-Supervised and Masked Learning

Masked self-supervision has achieved strong results in vision and language by masking parts of the input and training models to reconstruct them, which encourages learning global dependencies rather than memorizing local statistics [Devlin et al., 2019, He et al., 2022]. Recent work has also applied masked autoencoders to scRNA-seq for representation learning [Fang et al., 2024]. However, in scRNA-seq the distinction between an observed zero and a missing value is itself uncertain, and naive reconstruction objectives can encourage over-imputation of genuine biological zeros [Sarkar and Stephens, 2021, Linderman et al., 2022b]. MaskImpute adapts masking to this setting by (i) using different masking rates for zeros and non-zeros, and (ii) incorporating a biozero-aware prior and regularization so that reconstruction pressure is reduced on zeros likely to be biological non-expression.

3 Methods

3.1 Synthetic scRNA-seq Datasets

We benchmarked MaskImpute against other approaches on deterministic synthetic scRNA-seq datasets generated with the Splat simulator from the Splatter package [Zappia et al., 2017]. The goal is controlled evaluation in a setting where both the observed counts and pre-dropout ground truth are available.

Summary. We simulate 13 scenarios with $G = 1000$ genes spanning grouped populations, batch effects, and trajectory topologies. Each scenario is generated deterministically with fixed seeds and split into a tuning split ($N = 1,000$ cells), a held-out test split ($N = 5,000$ cells), and a runtime-scaling split ($N \in \{10,000, \dots, 100,000\}$) on a representative dropout scenario. Datasets store observed counts and pre-dropout ground truth, along with $\log_2(1+\text{CP10k})$ assays; marker genes are those with differential-expression factors not equal to 1 in any group. Full simulator parameters and the scenario catalog are provided in Appendix A.

Notation. In this work, $\mathbf{X}_{\log} \in \mathbb{R}_{\geq 0}^{N \times G}$ denotes the observed log-normalized matrix, $\mathbf{X}_{\log}^*$ the corresponding synthetic ground truth (when available), and $\mathbf{X}'_{\log}$ the reconstructed matrix produced by MaskImpute. We write x_{ij} , x_{ij}^* , and x'_{ij} for their respective entries. We denote the (unnormalized) count matrix by $\mathbf{C} = \{c_{ij}\}$.

3.2 Model Architecture

MaskImpute is a masked denoising autoencoder trained with a biozero-aware corruption and regularization scheme to reduce over-imputation at observed zeros. Let $\tilde{\mathbf{s}}_i \in \mathbb{R}^G$ denote the corrupted, scaled input for cell i (Sections 3.3–3.4). The encoder $f_\theta : \mathbb{R}^G \rightarrow \mathbb{R}^{d_{\text{lat}}}$ maps $\tilde{\mathbf{s}}_i$ to a latent representation $\mathbf{h}_i = f_\theta(\tilde{\mathbf{s}}_i)$, and a decoder $g_\phi : \mathbb{R}^{d_{\text{lat}}} \rightarrow \mathbb{R}^G$ produces the reconstruction

$$\hat{\mathbf{s}}_i = g_\phi(\mathbf{h}_i) = g_\phi(f_\theta(\tilde{\mathbf{s}}_i)).$$

We use a symmetric fully connected architecture with encoder widths $128 \rightarrow 64 \rightarrow d_{\text{lat}}$ and decoder widths $d_{\text{lat}} \rightarrow 64 \rightarrow 128 \rightarrow G$, with bottleneck dimension $d_{\text{lat}}=16$. Each hidden block applies a linear transformation, layer normalization, and a SiLU nonlinearity. This yields a compact latent representation while preserving enough capacity to model nonlinear gene–gene structure.

Let $\mathbf{S} = \text{scale}(\mathbf{X}_{\log})$ and $\hat{\mathbf{S}}$ denote the scaled inputs and reconstructions stacked across cells. Reconstructions in log space are obtained by inverse scaling,

$$\mathbf{X}'_{\log} = \text{invscale}(\hat{\mathbf{S}}).$$

All entries, including observed non-zeros, are reconstructed by the same model. This makes the method a full-matrix imputer rather than a zero-only editing procedure.

3.3 Gene Scaling

We apply a robust, monotone per-gene scaling with three steps:

$$\begin{aligned} \text{(Winsorize)} \quad \bar{x}_{ij} &= \min(\max(x_{ij}, q_j^{(2)}), q_j^{(99.5)}), \\ \text{(Standardize)} \quad z_{ij} &= \frac{\bar{x}_{ij} - \mu_j}{\sigma_j}, \quad \sigma_j \leftarrow \max(\sigma_j, \varepsilon), \\ \text{(Rescale)} \quad s_{ij} &= 2 \cdot \frac{z_{ij} - z_j^{\min}}{z_j^{\max} - z_j^{\min}} - 1, \\ &\quad z_j^{\max} - z_j^{\min} \geq \varepsilon. \end{aligned}$$

We denote the scaled matrix by $\mathbf{S}$ and use $\text{scale}(\cdot) / \text{invscale}(\cdot)$ for forward/inverse transforms. Here $\varepsilon > 0$ is an arbitrarily small constant used for numerical stability (we use $\varepsilon = 10^{-8}$). We define $z_j^{\min} = \min_i z_{ij}$ and $z_j^{\max} = \max_i z_{ij}$.

150 3.4 Masked Denoising Objective

151 For each mini-batch $\mathcal{B}$ of cells, we sample corruption masks with different rates for observed zeros
 152 vs. observed non-zeros. Let c_{ij} denote the observed count and let x_{ij} denote the corresponding log-
 153 normalized value. Define the observed-zero set $\mathcal{Z} = \{(i, j) : c_{ij} = 0\}$ and the observed non-zero
 154 set $\mathcal{N} = \{(i, j) : c_{ij} > 0\}$. We sample two Bernoulli masks:

$$\begin{aligned} m_{ij}^{(0)} &\sim \text{Bernoulli}(\pi_{ij}^{(0)}), & \pi_{ij}^{(0)} &= p_{\text{zero}} p_{\text{bio}}(i, j) & \text{for } (i, j) \in \mathcal{Z}, \\ m_{ij}^{(\text{nz})} &\sim \text{Bernoulli}(p_{\text{nz}}) & & & \text{for } (i, j) \in \mathcal{N}, \end{aligned}$$

155 where $p_{\text{bio}}(i, j)$ is the biozero probability for an observed zero (Section 3.5). We use $(p_{\text{zero}}, p_{\text{nz}}) =$
 156 $(0.01, 0.05)$.

157 Masked non-zeros are replaced by zero in the scaled space. Masked zeros are replaced by count-
 158 derived noise as described below. Unmasked entries are left unchanged.

159 **Noise for masked zeros.** For masked zero entries, we inject noise in log space using the observed
 160 count range per gene. Let $c_j^{\text{max}} = \max_i c_{ij}$. For each masked zero entry we sample

$$\eta_{ij} \sim \text{Uniform}(0, \nu), \quad \nu = 0.02, \quad \tilde{x}_{ij} = \log_2(1 + \eta_{ij} c_j^{\text{max}}),$$

161 then apply $\text{scale}(\cdot)$ to obtain the corrupted scaled input value.

162 **Loss.** Let $s_{ij} = [\text{scale}(\mathbf{X}_{\text{log}})]_{ij}$ and $\hat{s}_{ij}$ denote the reconstruction in the scaled space, and let
 163 $r_{ij} = \hat{s}_{ij} - s_{ij}$ be the residual. We optimize a weighted masked MSE:

$$\mathcal{L}_{\text{mask}} = \frac{\omega_{\text{bio}} \sum_{(i,j) \in \mathcal{Z}} m_{ij}^{(0)} r_{ij}^2 + \omega_{\text{nz}} \sum_{(i,j) \in \mathcal{N}} m_{ij}^{(\text{nz})} r_{ij}^2}{\omega_{\text{bio}} \sum_{(i,j) \in \mathcal{Z}} m_{ij}^{(0)} + \omega_{\text{nz}} \sum_{(i,j) \in \mathcal{N}} m_{ij}^{(\text{nz})}},$$

164 with weights $(\omega_{\text{bio}}, \omega_{\text{nz}}) = (1.0, 2.0)$.

165 We add two auxiliary reconstruction terms. The biozero regularizer encourages predicted values at
 166 likely biological zeros to be close to the gene-wise scaled zero value:

$$\mathcal{L}_{\text{bio}} = \frac{\sum_{(i,j) \in \mathcal{Z}} p_{\text{bio}}(i, j) (\hat{s}_{ij} - s_{0j})^2}{\sum_{(i,j) \in \mathcal{Z}} p_{\text{bio}}(i, j)}, \quad s_{0j} = [\text{scale}(\mathbf{0})]_j.$$

167 Because the model reconstructs the full matrix (including observed non-zeros), we also include an
 168 observed-expression reconstruction penalty over $\mathcal{N}$,

$$\mathcal{L}_{\text{nz}} = \frac{\sum_{(i,j) \in \mathcal{N}} r_{ij}^2}{|\mathcal{N}|}.$$

169 The final objective is

$$\mathcal{L} = \mathcal{L}_{\text{mask}} + \lambda_{\text{bio}} \mathcal{L}_{\text{bio}} + \lambda_{\text{nz}} \mathcal{L}_{\text{nz}},$$

170 with $(\lambda_{\text{bio}}, \lambda_{\text{nz}}) = (1.2, 0.75)$.

171 3.5 Biological Zero Probability Estimation

172 We estimate a prior probability $p_{\text{bio}}(i, j)$ that an observed zero corresponds to true biological non-
 173 expression. We use a negative-binomial model for latent true counts with an explicit dropout mech-
 174 anism. Let t_{ij} denote a latent (pre-dropout) count and d_{ij} a dropout indicator:

$$t_{ij} \sim \text{NB}(\mu_{ij}, \phi_j), \quad d_{ij} \sim \text{Bernoulli}(\delta_{ij}), \quad c_{ij} = (1 - d_{ij}) t_{ij}.$$

175 Let $f_{ij}(0) = \Pr(t_{ij} = 0 \mid \mu_{ij}, \phi_j)$ denote the NB zero probability. Then for an observed zero
 176 (noting that $c_{ij} = 0$ whenever $t_{ij} = 0$, regardless of dropout),

$$p_{\text{bio}}(i, j) = \Pr(t_{ij} = 0 \mid c_{ij} = 0) = \frac{f_{ij}(0)}{\delta_{ij} + (1 - \delta_{ij}) f_{ij}(0)}.$$

177 We set $p_{\text{bio}}(i, j) = 0$ when $c_{ij} > 0$.

178 We model μ_{ij} using a gene mean μ_j and a library-size factor,

$$\mu_{ij} = \ell_i \mu_j, \quad \ell_i = \frac{\sum_j c_{ij}}{\text{median}_k \left(\sum_j c_{kj} \right)}.$$

179 Dropout probabilities δ_{ij} are modeled as a smooth decreasing function of $\log \mu_{ij}$ using a fitted logistic curve to empirical zero rates, and gene-wise dispersion parameters ϕ_j are estimated using
180 a shrinkage moment-based estimator on observed counts. This prior is used strictly as a soft regularizer and as a modulation signal for zero masking; it is not assumed to identify ground-truth
181 biological zeros in real data.
182
183

184 3.6 Training Details

185 We train MaskImpute in a transductive self-supervised setting: the model is fit on the full observed
186 matrix and learns to reconstruct randomly corrupted entries. Unless otherwise specified, we optimize the objective in Section 3.4 with Adam (learning rate 10^{-4} , weight decay 10^{-4}), batch size 32,
187 for 400 epochs on GPU. Random seeds are fixed for reproducibility. Biozero probabilities $p_{\text{bio}}(i, j)$
188 are computed once from observed counts (Section 3.5) and held fixed during training. All hyperparameters are held constant across datasets and splits.
189
190

191 3.7 Reconstruction-Informed Zero Shrinkage

192 The autoencoder outputs a full reconstruction $\hat{\mathbf{X}}_{\log}$ (all entries, including observed non-zeros). To
193 reduce over-imputation at observed zeros, we apply a lightweight post-hoc shrinkage *only* on entries
194 with $c_{ij} = 0$. Let $\hat{u}_{ij} = 2^{\hat{x}_{ij}} - 1$ denote the reconstructed CP10k-normalized count scale corresponding to $\hat{x}_{ij}$. For each gene j , let $\tilde{u}_j$ be a robust reference level, taken as the median of $\hat{u}_{ij}$ over
195 cells where the gene is observed as expressed ($c_{ij} > 0$), with a fallback to the 75th percentile of
196 $\{\hat{u}_{ij}\}_i$ when needed. We define a reconstruction-derived biozero weight
197

$$w_{ij}^{(s)} = \frac{1}{1 + \hat{u}_{ij} / \max(\tilde{u}_j, \varepsilon)} \quad \text{for } c_{ij} = 0,$$

198 and set $w_{ij}^{(s)} = 0$ for $c_{ij} > 0$. The final imputed normalized counts are

$$\hat{u}_{ij}^{(\text{shrink})} = \hat{u}_{ij} (1 - \alpha w_{ij}^{(s)}) \quad \text{for } c_{ij} = 0,$$

199 with shrinkage strength $\alpha = 0.3$, while entries with $c_{ij} > 0$ are left unchanged by this step. We
200 report final reconstructions in log space as $x'_{ij} = \log_2(1 + \hat{u}_{ij}^{(\text{shrink})})$.

201 3.8 Evaluation Metrics

202 **(i) Reconstruction error (synthetic test split).** On the synthetic test split ($N = 5000$ cells per
203 scenario), we compare $\mathbf{X}'_{\log}$ to the ground truth $\mathbf{X}_{\log}^*$ (both in $\log_2(1 + \text{CP10k})$ space). We report
204 mean squared error (MSE) and mean absolute error (MAE),

$$\text{MSE} = \frac{1}{NG} \sum_{i,j} (x_{ij}^* - x'_{ij})^2, \quad \text{MAE} = \frac{1}{NG} \sum_{i,j} |x_{ij}^* - x'_{ij}|.$$

205 We also report subset metrics on (a) dropout-like entries $D = \{(i, j) : x_{ij} = 0 \wedge x_{ij}^* > 0\}$,
206 (b) biozero entries $Z_{\text{bio}} = \{(i, j) : x_{ij}^* = 0\}$, and (c) marker-gene entries $C = \{(i, j) : j \in \mathcal{J}_M\}$, where $\mathcal{J}_M$ is defined from DEFacGroup* (Section 3.1). For each subset, the metric is the
207 corresponding average over the subset; if a subset is empty we report it as NA. For methods that
208 output imputed counts rather than logcounts, we normalize the imputed count matrix to the dataset
209 targetSum using its *imputed* per-cell library sizes, and then apply the $\log_2(1 + x)$ transform; we
210 do not use the ground-truth library sizes (`libSizeTrue`). Unless otherwise specified, stochastic
211 methods are run for five random seeds and results are reported as mean $\pm$ standard deviation.
212

213 **(ii) Per-gene normalized RMSE (gNRMSE).** To reduce sensitivity to the scale of highly ex-
 214 pressed genes, we report per-gene normalized RMSE,

$$\text{gNRMSE} = \frac{1}{G} \sum_{j=1}^G \frac{\sqrt{\frac{1}{N} \sum_i (x_{ij}^* - x'_{ij})^2}}{\max(\text{sd}(\{x_{ij}^*\}_{i=1}^N), \epsilon)},$$

215 and its marker-gene variant, averaged over $j \in \mathcal{J}_M$.

216 **(iii) Runtime scaling.** To evaluate scalability, we measure wall-clock runtime on the scaling split
 217 for

$$N \in \{10000, 20000, \dots, 100000\}$$

218 on the representative scenario `groups_balanced_moderate_drop`. We report mean runtime (and
 219 standard deviation across datasets when applicable). All runtime benchmarks and reported runtimes
 220 were obtained on an NVIDIA DGX H100 system and all methods were benchmarked on the same
 221 hardware (Appendix E).

222 **(iv) Real-data label agreement.** For labeled real datasets, no pre-dropout ground truth is available.
 223 We therefore evaluate only whether reconstructed matrices preserve externally annotated cell types.
 224 We use the full Baron human pancreas dataset [Baron et al., 2016], the full Zeisel mouse cortex
 225 dataset [Zeisel et al., 2015], and the 10x PBMC68k dataset with the published barcode-level annota-
 226 tions from the original analysis [Zheng et al., 2017]. To avoid supervised marker selection, we select
 227 the top 1,000 highly variable genes from the observed matrix only, using mean-binned dispersion
 228 of log-normalized expression. Labels are used only after imputation for evaluation. For computa-
 229 tionally tractable repeated evaluation, PBMC68k is evaluated on a fixed random 12,000-cell subset.
 230 For each imputed matrix, we standardize genes, compute PCA embeddings with $d \in \{20, 50, 100\}$
 231 components, build a shared nearest-neighbor graph with $k \in \{10, 15, 30\}$, and run Leiden clustering
 232 [Traag et al., 2019] at resolutions $\{0.2, 0.5, 1.0, 1.5, 2.0\}$. We repeat the procedure for five seeds and
 233 report the mean and standard deviation across this grid. We also report a cluster-count-matched sum-
 234 mary that selects, for each seed, the grid point whose number of clusters is closest to the number of
 235 annotated cell types; this uses only the number of labels, not label identities. When batch-like meta-
 236 data are available (Baron donor, Zeisel cell-id prefix, PBMC barcode suffix), we report cluster-batch
 237 ARI as a diagnostic, where lower values indicate weaker alignment between clusters and batch. DCA
 238 is run with a 300-second wall-clock budget per dataset; timeouts are recorded explicitly rather than
 239 omitted. This evaluation remains a downstream sanity check rather than a direct denoising metric,
 240 and it can favor conservative methods when raw expression already separates annotated populations.

241 **(v) Synthetic downstream label agreement and calibration.** For the phase-6 diagnostic analyses,
 242 we also evaluate clustering on the synthetic test split using the simulator-provided group or path
 243 labels. Labels are used only for evaluation. We use the same PCA-SNN-Leiden procedure as in the
 244 real-data analysis, but with a smaller robustness grid $d \in \{20, 50\}$, $k \in \{10, 15, 30\}$, and resolutions
 245 $\{0.2, 0.5, 1.0\}$. We report both grid-averaged ARI/NMI and a cluster-count-matched summary. We
 246 additionally examine a one-parameter, label-free residual calibration

$$\mathbf{X}_{\log}^{(\lambda)} = (1 - \lambda)\mathbf{X}'_{\log} + \lambda\mathbf{X}_{\log},$$

247 where $\mathbf{X}_{\log}$ is the observed log-normalized matrix. This transform does not use labels or dataset-
 248 specific fitting and is evaluated only as a simple calibration of the final output.

249 3.9 Algorithm Outline

250 Figure 1 provides a high-level view of the MaskImpute pipeline, from biozero probability estimation
 251 to masked corruption and reconstruction. Algorithm 1 gives the complete training and imputation
 252 procedure, including mask sampling, corruption, and the loss minimized by MaskImpute.

Algorithm 1 MaskImpute training and imputation.

- 1: **Input:** logcounts $\mathbf{X}_{\log}$, counts $\mathbf{C}$, scaler $\text{scale}/\text{invscale}$, masking rates $p_{\text{zero}}, p_{\text{nz}}$, noise level ν , shrinkage strength α , and epochs E .
 - 2: Compute biozero probabilities $p_{\text{bio}}(i, j)$ from counts via Section 3.5.
 - 3: Scale $\mathbf{S} \leftarrow \text{scale}(\mathbf{X}_{\log})$.
 - 4: **for** epoch = 1 to E **do**
 - 5: **for** mini-batch $\mathcal{B}$ **do**
 - 6: Sample masks on entries: $m_{ij}^{(0)} \sim \text{Bernoulli}(p_{\text{zero}}p_{\text{bio}}(i, j))$ for observed zeros ($c_{ij} = 0$), and $m_{ij}^{(\text{nz})} \sim \text{Bernoulli}(p_{\text{nz}})$ for observed non-zeros ($c_{ij} > 0$).
 - 7: Corrupt inputs: masked non-zeros $\tilde{s}_{ij} \leftarrow 0$; masked zeros $\tilde{x}_{ij} = \log_2(1 + \eta_{ij}c_j^{\max})$ with $\eta_{ij} \sim \text{Uniform}(0, \nu)$ and $c_j^{\max} = \max_i c_{ij}$, then $\tilde{s}_{ij} \leftarrow \text{scale}(\tilde{x}_{ij})$.
 - 8: Forward pass: $\hat{s}_i = g_{\phi}(f_{\theta}(\tilde{\mathbf{s}}_i))$.
 - 9: Update parameters by minimizing $\mathcal{L}$ (Section 3.4).
 - 10: **end for**
 - 11: **end for**
 - 12: Reconstruct $\hat{\mathbf{X}}_{\log} \leftarrow \text{invscale}(\hat{\mathbf{S}})$.
 - 13: Apply reconstruction-informed shrinkage on observed zeros in the normalized count scale (Section 3.7) to obtain $\mathbf{X}'_{\log}$.
 - 14: **Output:** reconstructed logcounts $\mathbf{X}'_{\log}$.
-

253 4 Experiments and Results

254 4.1 Denoising Accuracy on Synthetic Data

255 We evaluate imputation accuracy on the 13 synthetic scenarios described in Section 3.1 on the held-
256 out test split ($N = 5,000$ cells per scenario). We compare reconstructed logcounts $\mathbf{X}'_{\log}$ against the
257 synthetic ground truth $\mathbf{X}_{\log}^*$ using the metrics in Section 3.8. In addition to overall reconstruction
258 error, we report subset errors on dropout-like entries (observed zeros with non-zero ground truth),
259 biozero entries (ground-truth zeros), and marker-gene entries. Unless otherwise stated, we run each
260 method for 5 repeats with different random seeds per scenario. Baselines with available generated
261 results include DCA, scVI, ALRA, MAGIC, AutoClass, cclmpute, SAVER, and the un-denoised
262 observed logcounts (Baseline). scVI is run with a negative-binomial likelihood, two hidden layers,
263 latent dimension 10, and 400 training epochs; its normalized expression output is scaled to the
264 benchmark CP10k target before log transformation. ALRA is run as a fixed adaptive low-rank
265 reconstruction baseline on log-normalized expression, with rank selected from the singular-value
266 spectrum and gene-wise thresholding of low reconstructed values.

267 Table 1 summarizes average performance across scenarios. Among the baselines with available
268 results, MaskImpute achieves the lowest mean MSE and dropout-MSE. Relative to DCA, MaskIm-
269 pute reduces overall MSE by 38% and dropout-MSE by 34% in this benchmark; relative to scVI,
270 it reduces overall MSE by 35% and dropout-MSE by 32%. Marker-gene subset errors are reported
271 in Appendix B (Table 3). MaskImpute also has lower mean Biozero-MSE than DCA (0.0133 vs.
272 0.0164) and scVI (0.0133 vs. 0.0179), but the DCA comparison is not uniform across scenarios.
273 Biozero-focused errors are lowest for conservative strategies that preserve observed zeros (Baseline,
274 SAVER), which is expected because biozero entries have ground-truth zero expression. MaskImpute
275 therefore prioritizes dropout recovery and full-matrix accuracy while maintaining Biozero-MSE be-
276 low the neural baselines. We report per-scenario MSE and marker-gene subset errors in Appendix B,
277 along with scenario-level wins and paired Wilcoxon signed-rank tests against DCA.

278 We note that gNRMSE values can exceed 1 because the metric normalizes by each gene’s standard
279 deviation, including low-variance genes where even small absolute errors translate into large relative
280 error. In our benchmark, MaskImpute still improves gNRMSE relative to the baselines with available
281 results (Table 1). Appendix B (Figure 2) summarizes benchmark metrics aggregated across scenarios
282 (mean $\pm$ standard deviation).

Table 1: Synthetic test split benchmark summary across 13 scenarios and 5 seeds. Entries report means across scenario-level estimates; $\pm$ values are standard deviations across scenarios. Each scenario-level estimate averages 5 random seeds. Baseline denotes the un-denoised observed log-counts (identity). Lower is better.

Method	MSE	Dropout-MSE	Biozero-MSE	MAE	gNRMSE
MaskImpute	0.2242 $\pm$ 0.0399	0.3053 $\pm$ 0.0374	0.0133 $\pm$ 0.0055	0.3485 $\pm$ 0.0320	1.1592 $\pm$ 0.0870
DCA	0.3598 $\pm$ 0.1306	0.4652 $\pm$ 0.1373	0.0164 $\pm$ 0.0243	0.4504 $\pm$ 0.0879	1.3867 $\pm$ 0.2161
scVI	0.3426 $\pm$ 0.1194	0.4511 $\pm$ 0.1292	0.0179 $\pm$ 0.0225	0.4355 $\pm$ 0.0789	1.3502 $\pm$ 0.1948
ALRA	0.6917 $\pm$ 0.2917	0.7912 $\pm$ 0.2646	0.0103 $\pm$ 0.0205	0.6498 $\pm$ 0.1566	1.8498 $\pm$ 0.3737
MAGIC	0.6411 $\pm$ 0.2850	0.7796 $\pm$ 0.2693	0.0104 $\pm$ 0.0195	0.6256 $\pm$ 0.1557	1.7747 $\pm$ 0.3693
AutoClass	0.7281 $\pm$ 0.3038	0.8621 $\pm$ 0.2971	0.0098 $\pm$ 0.0204	0.6721 $\pm$ 0.1593	1.9457 $\pm$ 0.3888
ccImpute	0.6983 $\pm$ 0.4751	1.4643 $\pm$ 0.6006	0.0095 $\pm$ 0.0308	0.5068 $\pm$ 0.2240	1.6406 $\pm$ 0.6393
SAVER	1.2208 $\pm$ 0.5005	2.6786 $\pm$ 0.3557	0.0020 $\pm$ 0.0064	0.6745 $\pm$ 0.2190	2.4384 $\pm$ 0.7013
Baseline	1.5236 $\pm$ 0.5768	3.3665 $\pm$ 0.3281	0.0000 $\pm$ 0.0000	0.7343 $\pm$ 0.2569	2.7044 $\pm$ 0.8656

4.2 Ablations, Calibration, and Real-Data Checks

Appendix C reports component ablations, sensitivity analyses, and a label-free residual-calibration sweep. The ablations show that the biozero regularizer is the primary component protecting biological zeros: removing it reduces aggregate MSE but increases Biozero-MSE from 0.013 to 0.095. Removing zero shrinkage has a smaller but consistent effect, increasing Biozero-MSE to 0.018. A simple residual calibration $X^{(\lambda)} = (1 - \lambda)X_{\text{MI}} + \lambda X_{\text{obs}}$ can improve the diagnostic efficacy score; $\lambda = 0.05$ is the safest synthetic-error trade-off, while $\lambda = 0.10$ gives the strongest synthetic downstream ARI/NMI in the screened grid.

Appendix D gives downstream label-agreement results on Baron human pancreas, Zeisel mouse cortex, and PBMC68k. These real-data experiments do not provide pre-dropout ground truth and are therefore reported as structure-preservation checks rather than direct imputation-accuracy evidence. MaskImpute has the best cluster-count-matched Baron ARI/NMI among evaluated methods and the best matched Zeisel NMI among imputation methods, but PBMC68k does not show a MaskImpute clustering advantage. The synthetic ground-truth benchmark remains the primary evidence for imputation accuracy.

5 Limitations

MaskImpute introduces additional hyperparameters (masking rates, regularization, shrinkage strength) whose interactions may be dataset-dependent; the sensitivity analysis in Section 4.2 is diagnostic and should be expanded to additional datasets before final deployment. The count-derived biozero prior is used only as a soft regularizer and masking modulator, and may be misspecified for some protocols or preprocessing pipelines. The real-data experiments use annotated cell-type labels as a downstream proxy, not ground-truth molecular reconstructions, and therefore can reward conservative behavior on marker-rich datasets. Finally, the current runtime results are based on an unoptimized implementation; we expect improved scalability with mini-batch inference and mixed-precision training, but these optimizations must be empirically validated. In terms of broader impact, improved denoising can support biological discovery and more stable downstream analyses, but over-imputation or misuse of denoised matrices as “ground truth” can lead to spurious conclusions; we therefore recommend reporting sensitivity to key hyperparameters and validating findings against raw data and orthogonal assays when possible.

6 Software and Data Availability

For double-blind review, links to code and any processed artifacts are redacted. An anonymized zip archive containing code, preprocessing scripts, and experiment configurations is provided as a separate file with this submission for reviewers. We will release the same materials publicly upon acceptance. Synthetic datasets are generated with Splatter [Zappia et al., 2017]. The anonymized supplementary archive includes the code, configuration files, dependency notes, and license information for released scripts and third-party assets used to reproduce the reported experiments.

References

- Maayan Baron, Adrian Veres, Samuel L. Wolock, Aubrey L. Faust, Renaud Gaujoux, Amedeo Vetere, Jennifer Hyoje Ryu, Bridget K. Wagner, Shai S. Shen-Orr, Allon M. Klein, Douglas A. Melton, and Itai Yanai. A single-cell transcriptomic map of the human and mouse pancreas reveals inter- and intra-cell population structure. *Cell Systems*, 3(4):346–360.e4, 2016. doi: 10.1016/j.cels.2016.08.011.
- Philip Brennecke, Simon Anders, Jong K. Kim, et al. Accounting for technical noise in single-cell RNA-seq experiments. *Nature Methods*, 10(11):1093–1095, 2013. doi: 10.1038/nmeth.2645. URL <https://doi.org/10.1038/nmeth.2645>.
- Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, pages 4171–4186, 2019. doi: 10.18653/v1/N19-1423. URL <https://doi.org/10.18653/v1/N19-1423>.
- Gökçen Eraslan, Lukas M. Simon, Maria Mircea, Nikola S. Mueller, and Fabian J. Theis. Single-cell RNA-seq denoising using a deep count autoencoder. *Nature Communications*, 10:390, 2019. doi: 10.1038/s41467-018-07931-2. URL <https://doi.org/10.1038/s41467-018-07931-2>.
- Wanxin Fang, Cameron M. Bell, Adam Sapirstein, Shinnosuke Asami, Noah J. Leeper, et al. scMAE: a masked autoencoder for single-cell RNA-seq clustering. *Bioinformatics*, 40(1):btae020, 2024. doi: 10.1093/bioinformatics/btae020. URL <https://doi.org/10.1093/bioinformatics/btae020>.
- Kaiming He, Xinlei Chen, Saining Xie, Yanghao Li, Piotr Dollár, and Ross Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 16000–16009, 2022.
- Stephanie C. Hicks, F. William Townes, Mingxiang Teng, and Rafael A. Irizarry. Missing data and technical variability in single-cell RNA-sequencing experiments. *Biostatistics*, 19(4):562–578, 2018. doi: 10.1093/biostatistics/kxx053. URL <https://doi.org/10.1093/biostatistics/kxx053>.
- Wenpin Hou, Zhicheng Ji, Hongkai Ji, and Stephanie C. Hicks. A systematic evaluation of single-cell RNA-sequencing imputation methods. *Genome Biology*, 21:218, 2020. doi: 10.1186/s13059-020-02132-x. URL <https://doi.org/10.1186/s13059-020-02132-x>.
- Mo Huang, Jingshu Wang, Eduardo Torre, et al. SAVER: gene expression recovery for single-cell RNA sequencing. *Nature Methods*, 15:539–542, 2018. doi: 10.1038/s41592-018-0033-z. URL <https://doi.org/10.1038/s41592-018-0033-z>.
- Ruochen Jiang, Tianyi Sun, Dongyuan Song, and Jingyi Jessica Li. Statistics or biology: the zero-inflation controversy about scRNA-seq data. *Genome Biology*, 23(1):31, 2022. doi: 10.1186/s13059-022-02601-5.
- Vladimir Y. Kiselev, Kristina Kirschner, Michael T. Schaub, Tallulah Andrews, Andrew Yiu, Tamir Chandra, Kedar N. Natarajan, Wolf Reik, Mauricio Barahona, Anthony R. Green, and Martin Hemberg. Sc3: consensus clustering of single-cell RNA-seq data. *Nature Methods*, 14(5):483–486, 2017. doi: 10.1038/nmeth.4236.
- Hui Li, Weijun Luo, et al. A universal deep neural network for in-depth cleaning of single-cell RNA-seq data. *Nature Communications*, 13:1901, 2022. doi: 10.1038/s41467-022-29576-y. URL <https://doi.org/10.1038/s41467-022-29576-y>.
- Wei Vivian Li and Jingyi Jessica Li. An accurate and robust imputation method scImpute for single-cell RNA-seq data. *Nature Communications*, 9:997, 2018. doi: 10.1038/s41467-018-03405-7. URL <https://doi.org/10.1038/s41467-018-03405-7>.
- George C. Linderman, Jun Zhao, Manolis Roulis, Piotr Bielecki, Richard A. Flavell, Boaz Nadler, and Yuval Kluger. Zero-preserving imputation of single-cell RNA-seq data. *Nature Communications*, 13:192, 2022a. doi: 10.1038/s41467-021-27729-z.

George C. Linderman, Jun Zhao, Manolis Roulis, Piotr Bielecki, Richard A. Flavell, Boaz Nadler, and Yuval Kluger. Zero-preserving imputation of single-cell RNA-seq data. *Nature Communications*, 13:192, 2022b. doi: 10.1038/s41467-021-27729-z. URL <https://doi.org/10.1038/s41467-021-27729-z>.

Romain Lopez, Jeffrey Regier, Michael B. Cole, Michael I. Jordan, and Nir Yosef. Deep generative modeling for single-cell transcriptomics. *Nature Methods*, 15:1053–1058, 2018. doi: 10.1038/s41592-018-0229-2.

Malte D. Luecken and Fabian J. Theis. Current best practices in single-cell RNA-seq analysis: a tutorial. *Molecular Systems Biology*, 15(6):e8746, 2019. doi: 10.15252/msb.20188746. URL <https://doi.org/10.15252/msb.20188746>.

Marcin Malec, Hasan Kurban, and Mehmet Dalkilic. ccimpute: an accurate and scalable consensus clustering based algorithm to impute dropout events in the single-cell rna-seq data. *BMC Bioinformatics*, 23:291, 2022. doi: 10.1186/s12859-022-04814-8.

Abhishek Sarkar and Matthew Stephens. Separating measurement and expression models clarifies confusion in single-cell RNA sequencing analysis. *Nature Genetics*, 53:770–777, 2021. doi: 10.1038/s41588-021-00873-4. URL <https://doi.org/10.1038/s41588-021-00873-4>.

Valentine Svensson. Droplet scRNA-seq is not zero-inflated. *Nature Biotechnology*, 38:147–150, 2020. doi: 10.1038/s41587-019-0379-5. URL <https://doi.org/10.1038/s41587-019-0379-5>.

Vincent A. Traag, Ludo Waltman, and Nees Jan van Eck. From louvain to leiden: guaranteeing well-connected communities. *Scientific Reports*, 9:5233, 2019. doi: 10.1038/s41598-019-41695-z.

Bram Van de Sande et al. Applications of single-cell RNA sequencing in drug discovery and development. *Nature Reviews Drug Discovery*, 2023. doi: 10.1038/s41573-023-00688-4. URL <https://doi.org/10.1038/s41573-023-00688-4>.

David van Dijk, Roshan Sharma, Juozas Nainys, et al. Recovering gene interactions from single-cell data using data diffusion. *Cell*, 174(3):716–729.e27, 2018. doi: 10.1016/j.cell.2018.05.061. URL <https://doi.org/10.1016/j.cell.2018.05.061>.

Luke Zappia, Belinda Phipson, and Alicia Oshlack. Splatter: simulation of single-cell rna sequencing data. *Genome biology*, 18(1):174, 2017.

Amit Zeisel, Ana B. Muñoz-Manchado, Simone Codeluppi, Peter Lönnerberg, Gioele La Manno, Anna Juréus, Sueli Marques, Hermany Munguba, Liquan He, Christer Betsholtz, Charlotte Rolny, Gonçalo Castelo-Branco, Jens Hjerling-Leffler, and Sten Linnarsson. Brain structure, cell types in the mouse cortex and hippocampus revealed by single-cell RNA-seq. *Science*, 347(6226):1138–1142, 2015. doi: 10.1126/science.aaa1934.

Grace X. Y. Zheng, Jessica M. Terry, Phillip Belgrader, Paul Ryvkin, Zachary W. Bent, Ryan Wilson, Solongo B. Ziraldo, Tobias D. Wheeler, Geoff P. McDermott, Junjie Zhu, and others. Massively parallel digital transcriptional profiling of single cells. *Nature Communications*, 8:14049, 2017. doi: 10.1038/ncomms14049.

A Synthetic Simulation Details

This appendix describes the data-generation protocol underlying Table 1 and Appendix Tables 2–5. It summarizes the fixed train/test/runtime splits, the 13 scenario definitions, the grouped, batch-effect, and trajectory simulation settings, and the assays and marker-gene annotations used by the benchmark metrics.

Global settings and splits. All simulations start with $G = 1000$ genes and are filtered to retain genes expressed in at least three observed cells. We generate three splits: (i) a tuning split with $N = 1000$ cells per scenario, (ii) a held-out test split with $N = 5000$ cells per scenario, and (iii) a runtime-scaling split with $N \in \{10000, 20000, \dots, 100000\}$ on a single representative scenario (groups_balanced_moderate_drop). All datasets are generated with fixed seeds (split-specific offsets), so each scenario corresponds to one deterministic dataset per split. We use the tuning split only for model selection and hyperparameter choices; all reported reconstruction metrics are computed on the held-out test split.

Scenario catalog. The benchmark contains 13 scenarios: one no-dropout baseline (groups_balanced_nodrop) and six scenario families, each with an experiment-level dropout variant and a group-level dropout variant: balanced groups (groups_balanced_moderate_drop), imbalanced groups (groups_imbalanced_moderate_drop), rare populations (groups_rare_high_drop), batch effects (batch_effects_moderate_drop), linear trajectories (paths_linear_moderate_drop), and branching trajectories (paths_branching_moderate_drop). Group-level variants append _group to the scenario id.

Grouped population scenarios. Grouped datasets use Splat’s groups mode with group proportions (group.prob): balanced (1/3, 1/3, 1/3), imbalanced (0.6, 0.3, 0.1), and rare (0.50, 0.25, 0.20, 0.05). We fix the fraction of differentially expressed genes (de.prob) to 0.10 in all grouped scenarios. Dropout is simulated using Splatter’s logistic dropout model, which maps log mean expression to a dropout probability via a logistic function with midpoint (dropout.mid) and slope (dropout.shape); we use moderate-dropout settings dropout.mid = 3.0 and dropout.shape = -1.0, and rare high-dropout settings dropout.mid = 4.0 and dropout.shape = -1.0. The scope of dropout heterogeneity is controlled by dropout.type: experiment-level variants use dropout.type=experiment (shared parameters), while group-level variants use dropout.type=group with group-specific parameter vectors created by evenly perturbing the base values across groups (dropout.mid range ± 0.4 , dropout.shape range ± 0.25).

Batch-effect scenario. The batch-effect family uses two batches of approximately equal size (batchCells) and introduces batch-specific multiplicative gene factors via the batch effect location/scale parameters (batch.facLoc=0.10, batch.facScale=0.10). We also increase library-size dispersion (lib.scale=0.40) to induce greater heterogeneity in per-cell sequencing depth.

Trajectory scenarios. Trajectory datasets are generated with Splat’s paths mode. Both linear and branching trajectories use four groups with equal proportions (0.25, 0.25, 0.25, 0.25), a higher fraction of DE genes (de.prob=0.20), and stronger DE factor magnitudes via the log-normal location parameter (de.facLoc=0.20). We set the number of interpolation steps (path.nSteps) to 50. The trajectory topology is controlled by path.from: the linear topology uses path.from = (0, 1, 2, 3), while the branching topology uses path.from = (0, 1, 1, 3). Both trajectory families use the same moderate dropout settings as the grouped moderate-dropout scenarios, again with experiment-level (dropout.type=experiment) and group-level (dropout.type=group) dropout variants.

Assays, normalization, and marker genes. Each dataset stores observed counts (counts) and pre-dropout ground truth (TrueCounts), as well as CP10k-normalized log-transformed assays $\logcounts = \log_2(1 + \text{CP10k}(\text{counts}))$ and $\log\text{TrueCounts} = \log_2(1 + \text{CP10k}(\text{TrueCounts}))$, where observed and true counts are normalized independently. Per-cell observed and true library sizes and the normalization target sum (targetSum) are stored in metadata. Marker genes are defined using DEFacGroup* row-data columns: a gene is a marker if its differential-expression factor is not equal to 1 in any group.

457 B Per-Scenario Synthetic Results

458 This appendix expands the aggregate synthetic benchmark in Table 1. Table 2 reports per-scenario
 459 overall MSE for each method, Table 3 summarizes marker-gene reconstruction errors, Figure 2
 460 visualizes the main synthetic metrics across methods, Table 4 reports scenario-level wins by metric,
 461 and Table 5 gives paired Wilcoxon tests comparing MaskImpute with DCA across scenarios.

Table 2: Per-scenario overall MSE on the synthetic test split. Entries report mean $\pm$ repeat standard deviation over 5 seeds. Lower is better.

Scenario	MaskImpute	DCA	scVI	ALRA	MAGIC	AutoClass	ccImpute	SAVER	Baseline
batch_effects_moderate_drop	0.2362 $\pm$ 0.0008	0.3505 $\pm$ 0.0005	0.3290 $\pm$ 0.0103	0.6831 $\pm$ 0.0000	0.6083 $\pm$ 0.0005	0.7074 $\pm$ 0.0078	0.6042 $\pm$ 0.0037	1.2058 $\pm$ 0.0002	1.5365 $\pm$ 0.0000
batch_effects_moderate_drop_group	0.2216 $\pm$ 0.0039	0.3243 $\pm$ 0.0005	0.3053 $\pm$ 0.0078	0.6628 $\pm$ 0.0000	0.5815 $\pm$ 0.0005	0.6794 $\pm$ 0.0040	0.5640 $\pm$ 0.0009	1.2284 $\pm$ 0.0002	1.5863 $\pm$ 0.0000
groups_balanced_moderate_drop	0.2184 $\pm$ 0.0033	0.3428 $\pm$ 0.0004	0.3266 $\pm$ 0.0101	0.6444 $\pm$ 0.0000	0.6048 $\pm$ 0.0002	0.6844 $\pm$ 0.0059	0.6073 $\pm$ 0.0036	1.1731 $\pm$ 0.0002	1.4896 $\pm$ 0.0000
groups_balanced_moderate_drop_group	0.2104 $\pm$ 0.0023	0.3263 $\pm$ 0.0004	0.3148 $\pm$ 0.0087	0.6357 $\pm$ 0.0000	0.5750 $\pm$ 0.0002	0.6685 $\pm$ 0.0064	0.5516 $\pm$ 0.0006	1.1484 $\pm$ 0.0002	1.4636 $\pm$ 0.0000
groups_balanced_nodrop	0.1480 $\pm$ 0.0004	0.1488 $\pm$ 0.0004	0.1571 $\pm$ 0.0027	0.1470 $\pm$ 0.0000	0.1386 $\pm$ 0.0000	0.1518 $\pm$ 0.0012	0.0149 $\pm$ 0.0000	0.0208 $\pm$ 0.0000	0.0000 $\pm$ 0.0000
groups_imbalanced_moderate_drop	0.2131 $\pm$ 0.0023	0.3270 $\pm$ 0.0004	0.3176 $\pm$ 0.0114	0.6413 $\pm$ 0.0000	0.6036 $\pm$ 0.0003	0.6938 $\pm$ 0.0054	0.5895 $\pm$ 0.0030	1.2456 $\pm$ 0.0001	1.6054 $\pm$ 0.0000
groups_imbalanced_moderate_drop_group	0.2092 $\pm$ 0.0014	0.2907 $\pm$ 0.0002	0.2824 $\pm$ 0.0063	0.5413 $\pm$ 0.0000	0.4840 $\pm$ 0.0004	0.5775 $\pm$ 0.0063	0.4191 $\pm$ 0.0008	0.9851 $\pm$ 0.0003	1.3147 $\pm$ 0.0000
groups_rare_high_drop	0.3084 $\pm$ 0.0058	0.6620 $\pm$ 0.0010	0.6261 $\pm$ 0.0170	1.3608 $\pm$ 0.0000	1.3149 $\pm$ 0.0005	1.4241 $\pm$ 0.0053	1.8775 $\pm$ 0.0041	2.2323 $\pm$ 0.0001	2.5725 $\pm$ 0.0000
groups_rare_high_drop_group	0.2947 $\pm$ 0.0043	0.5961 $\pm$ 0.0007	0.5543 $\pm$ 0.0136	1.1689 $\pm$ 0.0000	1.1063 $\pm$ 0.0003	1.2197 $\pm$ 0.0036	1.5066 $\pm$ 0.0033	1.8991 $\pm$ 0.0003	2.2158 $\pm$ 0.0000
paths_branching_moderate_drop	0.2197 $\pm$ 0.0020	0.3316 $\pm$ 0.0005	0.3184 $\pm$ 0.0029	0.6305 $\pm$ 0.0000	0.5930 $\pm$ 0.0004	0.6700 $\pm$ 0.0039	0.5823 $\pm$ 0.0033	1.2059 $\pm$ 0.0001	1.5366 $\pm$ 0.0000
paths_branching_moderate_drop_group	0.2153 $\pm$ 0.0023	0.3169 $\pm$ 0.0011	0.2961 $\pm$ 0.0049	0.6424 $\pm$ 0.0000	0.5757 $\pm$ 0.0002	0.6853 $\pm$ 0.0084	0.5893 $\pm$ 0.0072	1.2605 $\pm$ 0.0002	1.6292 $\pm$ 0.0000
paths_linear_moderate_drop	0.2129 $\pm$ 0.0026	0.3311 $\pm$ 0.0003	0.3146 $\pm$ 0.0060	0.6210 $\pm$ 0.0000	0.5849 $\pm$ 0.0008	0.6621 $\pm$ 0.0066	0.5937 $\pm$ 0.0080	1.1616 $\pm$ 0.0001	1.4664 $\pm$ 0.0000
paths_linear_moderate_drop_group	0.2061 $\pm$ 0.0030	0.3289 $\pm$ 0.0008	0.3111 $\pm$ 0.0018	0.6131 $\pm$ 0.0000	0.5640 $\pm$ 0.0003	0.6419 $\pm$ 0.0033	0.5780 $\pm$ 0.0043	1.1043 $\pm$ 0.0002	1.3901 $\pm$ 0.0000

Table 3: Marker-gene subset errors on the synthetic test split across 13 scenarios. Entries report means across scenario-level estimates; $\pm$ values are standard deviations across scenarios. Lower is better.

Method	Marker-MSE	Marker-MAE	Marker-gNRMSE
MaskImpute	0.2235 $\pm$ 0.0410	0.3470 $\pm$ 0.0331	1.1072 $\pm$ 0.0864
DCA	0.3603 $\pm$ 0.1296	0.4490 $\pm$ 0.0871	1.3265 $\pm$ 0.2055
scVI	0.3468 $\pm$ 0.1188	0.4364 $\pm$ 0.0775	1.2981 $\pm$ 0.1837
ALRA	0.6970 $\pm$ 0.2936	0.6471 $\pm$ 0.1560	1.7596 $\pm$ 0.3518
MAGIC	0.6440 $\pm$ 0.2867	0.6224 $\pm$ 0.1550	1.6879 $\pm$ 0.3479
AutoClass	0.6662 $\pm$ 0.0310	0.6383 $\pm$ 0.0245	1.7260 $\pm$ 0.0001
ccImpute	0.7454 $\pm$ 0.5446	0.5176 $\pm$ 0.2569	1.5869 $\pm$ 0.6903
SAVER	1.2286 $\pm$ 0.5043	0.6716 $\pm$ 0.2184	2.2907 $\pm$ 0.6536
Baseline	1.5151 $\pm$ 0.5781	0.7277 $\pm$ 0.2556	2.5166 $\pm$ 0.8063

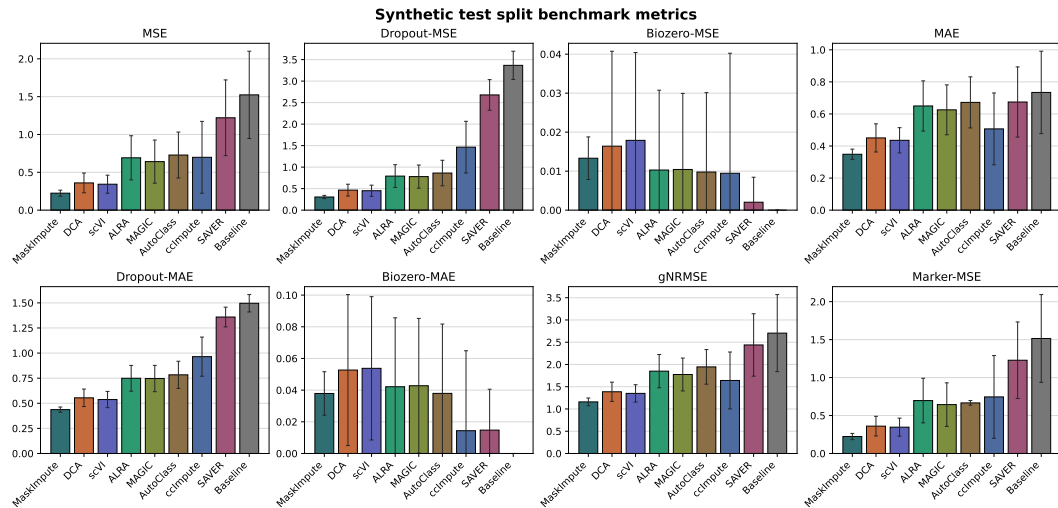


Figure 2: Benchmark metrics on the synthetic test split ($N=5,000$ cells per scenario). Each panel reports a metric aggregated across the 13 scenarios: bars show the mean across scenarios and error bars show the standard deviation, with lower error bars clipped at zero for non-negative metrics. Lower is better for all panels.

Table 4: Scenario-level wins on the synthetic test split. A win is assigned to the method with the lowest scenario-level mean for the metric.

Metric	Scenario-level wins
MSE	MaskImpute: 12/13; Baseline: 1/13
Dropout-MSE	MaskImpute: 12/12
Biozero-MSE	Baseline: 13/13
MAE	MaskImpute: 12/13; Baseline: 1/13
gNRMSE	MaskImpute: 12/13; Baseline: 1/13
Biozero-MAE	Baseline: 13/13
Marker-MSE	MaskImpute: 12/13; Baseline: 1/13
Marker-MAE	MaskImpute: 12/13; Baseline: 1/13
Marker-gNRMSE	MaskImpute: 12/13; Baseline: 1/13

Table 5: Paired Wilcoxon signed-rank tests comparing MaskImpute against DCA across synthetic test scenarios. The one-sided test uses the alternative that MaskImpute has lower error than DCA.

Metric	Wilcoxon two-sided p	Wilcoxon MaskImpute < DCA p
MSE	0.0002	0.0001
Dropout-MSE	0.0005	0.0002
Biozero-MSE	0.0215	0.9915
MAE	0.0002	0.0001
gNRMSE	0.0005	0.0002
Marker-MSE	0.0002	0.0001
Marker-MAE	0.0002	0.0001
Marker-gNRMSE	0.0005	0.0002

462 C Ablations, Calibration, and Synthetic Downstream Details

463 This appendix reports diagnostic experiments for the MaskImpute design. Table 6 isolates the con-
 464 tribution of major components, Table 7 evaluates residual calibration strengths, Table 8 reports syn-
 465 thetic downstream clustering diagnostics for selected outputs, and Table 9 summarizes one-scenario
 466 sensitivity to the shrinkage, biozero-weight, and zero mask-rate settings.

Table 6: MaskImpute component ablations on the synthetic test split. Entries are means across 13 scenarios; $\pm$ values are 95% confidence intervals across scenarios for efficacy, MSE, and Biozero-MSE. ARI and NMI are cluster-count-matched synthetic downstream scores. Lower is better for error metrics; higher is better for ARI/NMI.

Variant	Efficacy	MSE	Biozero-MSE	Dropout-MSE	ARI	NMI
MaskImpute	0.251 ± 0.017	0.224 ± 0.022	0.013 ± 0.003	0.304	0.718	0.764
No zero shrinkage	0.263 ± 0.017	0.228 ± 0.023	0.018 ± 0.004	0.312	0.716	0.762
No biozero regularization	0.360 ± 0.019	0.171 ± 0.008	0.095 ± 0.011	0.190	0.766	0.816
Uniform masking	0.319 ± 0.043	0.305 ± 0.047	0.007 ± 0.003	0.441	0.699	0.734
Plain masked AE	0.703 ± 0.146	0.682 ± 0.159	0.010 ± 0.011	0.792	0.639	0.676

Table 7: Label-free residual calibration on the synthetic test split. The calibrated output is $(1 - \lambda)X_{\text{MI}} + \lambda X_{\text{obs}}$. Entries are means across 13 scenarios. Lower is better for error metrics; higher is better for ARI/NMI.

Output	Efficacy	MSE	Biozero-MSE	Dropout-MSE	ARI	NMI
MaskImpute	0.251 ± 0.017	0.224	0.013	0.304	0.718	0.764
$0.95X_{\text{MI}} + 0.05X_{\text{obs}}$	0.243 ± 0.018	0.219	0.012	0.310	0.723	0.766
$0.90X_{\text{MI}} + 0.10X_{\text{obs}}$	0.243 ± 0.021	0.221	0.011	0.331	0.740	0.772
$0.85X_{\text{MI}} + 0.15X_{\text{obs}}$	0.250 ± 0.025	0.230	0.010	0.368	0.721	0.760
$0.80X_{\text{MI}} + 0.20X_{\text{obs}}$	0.264 ± 0.030	0.247	0.009	0.420	0.730	0.769
$0.75X_{\text{MI}} + 0.25X_{\text{obs}}$	0.287 ± 0.037	0.271	0.008	0.488	0.725	0.767
Observed input	1.524 ± 0.314	1.524	0.000	3.366	0.642	0.663

Table 8: Synthetic downstream clustering diagnostic for selected outputs. Matched columns select the clustering grid point with the closest number of clusters to the simulator-provided label count; grid columns average across the full PCA-SNN-Leiden grid. Higher is better.

Output	ARI (matched)	NMI (matched)	ARI (grid)	NMI (grid)
Observed input	0.642	0.663	0.543	0.585
MaskImpute	0.718	0.764	0.559	0.672
$0.95X_{\text{MI}} + 0.05X_{\text{obs}}$	0.723	0.766	0.565	0.674
$0.90X_{\text{MI}} + 0.10X_{\text{obs}}$	0.740	0.772	0.574	0.679

Table 9: One-scenario sensitivity analysis on groups_balanced_moderate_drop. Efficacy is $\text{MSE} + 2 \times \text{Biozero-MSE}$. Lower is better for error metrics; higher is better for ARI.

Parameter	Value	Efficacy	MSE	Biozero-MSE	Dropout-MSE	ARI
Shrinkage α	0	0.255	0.221	0.017	0.292	0.996
Shrinkage α	0.1	0.250	0.218	0.016	0.287	0.995
Shrinkage α	0.2	0.246	0.217	0.014	0.285	0.995
Shrinkage α	0.3	0.245	0.218	0.013	0.287	0.995
Shrinkage α	0.5	0.248	0.227	0.011	0.306	0.993
Shrinkage α	0.7	0.264	0.247	0.008	0.350	0.993
Shrinkage α	1	0.319	0.308	0.005	0.479	0.992
Biozero weight γ	0	0.361	0.166	0.097	0.184	0.989
Biozero weight γ	0.5	0.241	0.192	0.025	0.244	0.998
Biozero weight γ	1	0.242	0.211	0.015	0.276	0.996
Biozero weight γ	1.2	0.245	0.218	0.013	0.287	0.995
Biozero weight γ	2	0.260	0.243	0.008	0.328	0.995
Biozero weight γ	5	0.307	0.300	0.004	0.414	0.983
Zero mask rate p_0	0.005	0.244	0.216	0.014	0.284	0.996
Zero mask rate p_0	0.01	0.245	0.218	0.013	0.287	0.995
Zero mask rate p_0	0.02	0.246	0.222	0.012	0.293	0.995
Zero mask rate p_0	0.05	0.251	0.229	0.011	0.307	0.991

D Real-Data Label Agreement Details

This appendix contains the real-data label-agreement checks used as downstream diagnostics rather than direct imputation-accuracy evidence. Table 10 reports grid-averaged and cluster-count-matched ARI and NMI, together with batch ARI, for Baron, Zeisel, and PBMC68k after each imputation method.

Table 10: Real-data label agreement after imputation. ARI grid and NMI grid are means over five seeds and a PCA–SNN–Leiden robustness grid. Matched columns select the grid point whose number of clusters is closest to the number of annotated cell types for each seed. Higher is better for ARI and NMI. Batch ARI measures cluster–batch association, where lower is better.

Dataset	Method	ARI grid	NMI grid	ARI matched	NMI matched	Batch ARI
Baron	Observed	0.508 ± 0.170	0.771 ± 0.059	0.667 ± 0.098	0.830 ± 0.029	0.147 ± 0.040
Baron	DCA	0.442 ± 0.158	0.746 ± 0.062	0.607 ± 0.031	0.818 ± 0.005	0.160 ± 0.038
Baron	scVI	0.424 ± 0.139	0.739 ± 0.059	0.607 ± 0.031	0.816 ± 0.010	0.162 ± 0.037
Baron	ALRA	0.390 ± 0.155	0.721 ± 0.061	0.619 ± 0.031	0.811 ± 0.005	0.116 ± 0.033
Baron	MAGIC	0.315 ± 0.116	0.688 ± 0.053	0.523 ± 0.000	0.779 ± 0.000	0.156 ± 0.051
Baron	MaskImpute	0.456 ± 0.146	0.754 ± 0.059	0.684 ± 0.026	0.845 ± 0.007	0.163 ± 0.030
Zeisel	Observed	0.409 ± 0.174	0.629 ± 0.045	0.757 ± 0.002	0.738 ± 0.002	0.038 ± 0.005
Zeisel	DCA	0.381 ± 0.156	0.611 ± 0.035	0.655 ± 0.023	0.670 ± 0.012	0.042 ± 0.008
Zeisel	scVI	0.374 ± 0.152	0.604 ± 0.036	0.650 ± 0.014	0.675 ± 0.005	0.042 ± 0.007
Zeisel	ALRA	0.327 ± 0.126	0.610 ± 0.032	0.601 ± 0.015	0.676 ± 0.004	0.040 ± 0.005
Zeisel	MAGIC	0.248 ± 0.135	0.587 ± 0.040	0.623 ± 0.027	0.692 ± 0.009	0.055 ± 0.010
Zeisel	MaskImpute	0.366 ± 0.169	0.621 ± 0.042	0.688 ± 0.007	0.700 ± 0.009	0.042 ± 0.008
PBMC68k	Observed	0.214 ± 0.030	0.424 ± 0.049	0.216 ± 0.002	0.452 ± 0.022	0.000 ± 0.000
PBMC68k	DCA	0.212 ± 0.061	0.430 ± 0.052	0.277 ± 0.023	0.486 ± 0.017	0.001 ± 0.000
PBMC68k	scVI	0.216 ± 0.048	0.424 ± 0.049	0.254 ± 0.022	0.460 ± 0.009	0.000 ± 0.000
PBMC68k	ALRA	0.169 ± 0.054	0.412 ± 0.026	0.281 ± 0.032	0.462 ± 0.004	0.000 ± 0.000
PBMC68k	MAGIC	0.145 ± 0.045	0.374 ± 0.027	0.206 ± 0.001	0.449 ± 0.004	0.000 ± 0.000
PBMC68k	MaskImpute	0.187 ± 0.038	0.402 ± 0.030	0.220 ± 0.027	0.419 ± 0.009	0.000 ± 0.000

E Runtime Scaling Details

Runtime artifacts were generated on an NVIDIA DGX H100 system, with all methods in the runtime comparison benchmarked on the same hardware. Full CPU, RAM, and software-version details should be finalized with the post-feedback benchmark rerun. Figure 3 plots wall-clock scaling across synthetic cell-count settings, and Table 11 provides the corresponding runtime values in seconds for each method and cell count.

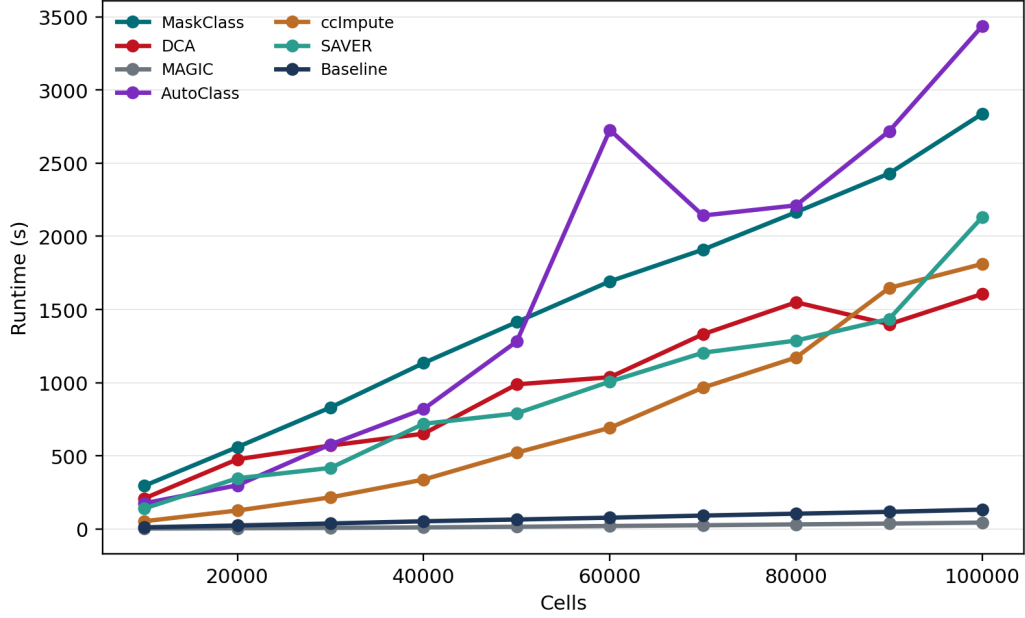


Figure 3: Runtime scaling on the synthetic scaling split (linear scale). Each point reports wall-clock runtime for a single dataset at the corresponding cell count.

Table 11: Runtime (seconds) for the synthetic scaling split.

Method	10000	20000	30000	40000	50000	60000	70000	80000	90000	100000
MaskImpute	295.15	559.69	830.76	1134.28	1414.03	1692.52	1909.26	2165.32	2430.20	2838.32
DCA	208.71	476.29	570.12	651.25	988.29	1037.25	1330.96	1549.41	1399.46	1606.54
MAGIC	2.77	4.98	7.15	10.61	15.04	20.31	25.26	31.02	36.74	43.53
AutoClass	175.62	298.51	577.81	819.63	1281.41	2727.90	2142.30	2212.01	2719.49	3438.38
ccImpute	52.90	126.05	215.93	337.61	522.05	691.23	966.11	1172.14	1647.80	1811.54
SAVER	139.83	346.51	417.33	718.73	790.14	1007.26	1204.63	1287.87	1435.66	2132.14
Baseline	11.10	23.73	37.34	52.66	64.58	77.05	91.47	104.35	116.88	132.26

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction explicitly state the method components and evaluation scope (synthetic and real scRNA-seq benchmarks).

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: See the Limitations section, which discusses hyperparameter sensitivity, prior misspecification, runtime/scalability, and risks of over-imputation or misuse.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include formal theoretical results (theorems/lemmas) requiring proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: We specify simulator configuration and splits (including full parameters in the appendix), model architecture/training hyperparameters, and evaluation metrics in the main paper and appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: An anonymized zip archive containing code and scripts is provided as a separate file with this submission for reviewers; the real datasets are publicly available from cited sources.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The paper details preprocessing, data splits, architecture, masking/regularization settings, optimizer, and training schedule; full simulation parameters are provided in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: Synthetic results are reported over multiple seeds as mean $\pm$ standard deviation and per-scenario tables are provided; additional paired tests are planned.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.

- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: All results were obtained on an NVIDIA DGX H100 system; runtime scaling plots and tables report wall-clock runtimes and are provided in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work uses publicly available, de-identified molecular datasets and does not involve new human subject recruitment or data collection.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The Limitations section briefly discusses potential benefits (supporting biological discovery) and risks (spurious conclusions from over-imputation or misuse of denoised matrices).

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out

that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.

- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper does not release high-risk generative models or scraped datasets; risks are limited to standard scientific misuse and are discussed as limitations.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: We cite the original datasets, simulator, and baseline methods; the anonymized supplementary archive contains dependency and license notes for released scripts and third-party assets.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The anonymized supplementary archive documents the released code, scripts, dependencies, expected inputs, and reproduction commands; no new human-subject dataset is introduced.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The work does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: The work does not involve new human subject studies; all evaluated datasets are publicly released and de-identified.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [N/A]

Justification: LLMs are not used as part of the proposed method; any use is limited to writing/editing and does not affect the scientific methodology or results.

Guidelines:

- 814 • The answer [N/A] means that the core method development in this research does not
- 815 involve LLMs as any important, original, or non-standard components.
- 816 • Please refer to our LLM policy in the NeurIPS handbook for what should or should
- 817 not be described.